(CSE 4712: Artificial Intelligence Lab)

Lab Report: Comparative analysis between BFS,DFS and UCS

Submitted by:
Md. Mosharrifur Rahman Ratul
Student ID: 210041213
Section: 2A
BSc in CSE, Dept of Computer Science and Engineering
Islamic University of Technology (IUT)

November 17, 2025

0.1 Task 3: State Space Search and Complexity Analysis

0.1.1 Overview

In this task, we implement and analyze uninformed search algorithms (BFS, DFS, UCS) for solving the N-Puzzle problem. The N-Puzzle consists of an $n \times n$ grid with numbered tiles and a blank space denoting 0. The objective is to reach a goal configuration by sliding tiles into the blank space.

0.1.2 Complexity Analysis

BFS

Here, in BFS , the algorithm searches the neighbour nodes first where the blank space (0) can be shifted. Then for each of the current state from the previous state, it checks whether it is the target state, if not it then creates its successor state around its neighbour nodes and continues for all the nodes in this level. Thus :-

- BFS ensures it always finds the shortest path or optimum moves for the solution of the puzzle.
- As it explores all the possible neighbour together, the memory complexity is not efficient; a large memory is required for complex initial puzzle state.
- Its time complexity could be $O(b^d)$. Here,
  b = branch number; range from 2(blanck space at the four corner of the puzzle) to 4(blank space in the middle)
  d = depth of solution, or number of average levels/-moves need to find the solution or targets

DFS

In DFS (Depth-First Search), the algorithm explores the deepest nodes first by following one path from the initial state as far as possible before backtracking. It continues exploring along one branch until it either reaches the goal or cannot go further, then backtracks to explore other branches. Thus:

- DFS is memory-efficient because it only stores a single path from the root to the current node, along with unexplored siblings.
- DFS does not guarantee the shortest path; it may find a solution far from optimal.
- Its time complexity can be $O(b^m)$, where
  b = branch number;range from 2 to 4
  m = maximum depth of the search space (longest path in the puzzle).
- *Space complexity is $O(m)$, which is much lower than BFS for large puzzles.*

UCS

Uniform-Cost Search (UCS) expands the node with the lowest total cost from the start. For the given N-Puzzle algorithm as every move has the same cost (1), UCS behaves similarly to BFS. The algorithm uses a priority queue to select the next node to expand based on cumulative cost. Thus:

- UCS always finds the least-cost solution, which is optimal when costs vary. Here, it is similar to BFS.
- Memory usage can be high because it stores all nodes in the frontier, similar to BFS.
- Time complexity is $O(b^d \log b^d)$, where
  b = branch number;range from 2 to 4

d = depth of the solution
the logarithmic factor comes from priority queue operations.

0.1.3 Sample Console Output

The following is a sample output of running the BFS algorithm on a 3x3 puzzle with the command :
python state_space_search.py –size 3 –start "1,2,3,4,5,0,6,7,8" –strategy "bfs"

```
Initial State (size=3):
1 2 3
4 5 _
6 7 8

Strategy: BFS
Open Nodes: 4503
Closed Nodes: 2827

Solution Path:
Step 0:
1 2 3
4 5 _
6 7 8

Step 1:
1 2 3
4 5 8
6 7 _

Step 2:
1 2 3
4 5 8
6 _ 7
```

```
Step 3:
1 2 3
4 5 8
_ 6 7

Step 4:
1 2 3
_ 5 8
4 6 7

Step 5:
1 2 3
5 _ 8
4 6 7

Step 6:
1 2 3
5 6 8
4 _ 7

Step 7:
1 2 3
5 6 8
4 7 _

Step 8:
1 2 3
5 6 _
4 7 8

Step 9:
1 2 3
5 _ 6
4 7 8

Step 10:
```

```
1 2 3
_ 5 6
4 7 8

Step 11:
1 2 3
4 5 6
_ 7 8

Step 12:
1 2 3
4 5 6
7 _ 8

Step 13:
1 2 3
4 5 6
7 8 _

Total steps: 13
```

The following is a sample output of running the UCS algorithm on a 3x3 puzzle with the command :
python state_space_search.py –size 3 –start "1,2,3,4,5,0,6,7,8" –strategy "ucs"

```

Initial State (size=3):
1 2 3
4 5 _
6 7 8

Strategy: UCS
Open Nodes: 3798
Closed Nodes: 2401

Solution Path:
Step 0:
```

```
1 2 3
4 5 _
6 7 8

Step 1:
1 2 3
4 5 8
6 7 _

Step 2:
1 2 3
4 5 8
6 _ 7

Step 3:
1 2 3
4 5 8
_ 6 7

Step 4:
1 2 3
_ 5 8
4 6 7

Step 5:
1 2 3
5 _ 8
4 6 7

Step 6:
1 2 3
5 6 8
4 _ 7

Step 7:
1 2 3
```

```
5 6 8
4 7 _

Step 8:
1 2 3
5 6 _
4 7 8

Step 9:
1 2 3
5 _ 6
4 7 8

Step 10:
1 2 3
_ 5 6
4 7 8

Step 11:
1 2 3
4 5 6
_ 7 8

Step 12:
1 2 3
4 5 6
7 _ 8

Step 13:
1 2 3
4 5 6
7 8 _

Total steps: 13
```

The following is a sample output of running the DFS algorithm on a 3x3 puzzle with the command :

python state_space_search.py –size 3 –start "1,2,3,4,5,0,6,7,8" –strategy "dfs"

```
Initial State (size=3):
1 2 3
4 5 _
6 7 8

Strategy: DFS
Open Nodes: 55086
Closed Nodes: 32129

Solution Path:
Step 0:
1 2 3
4 5 _
6 7 8

..
..
..
..
..

Step 31308:
1 2 3
4 5 _
7 8 6

Step 31309:
1 2 3
4 5 6
7 8 _

Total steps: 31309
```

BFS and UCS are similar, But DFS took a lot of step

to find the solution.